Q3: Terraform w zespole

Jak rozumieć pytanie, rozpoznawać scenariusze i oceniać odpowiedzi

Data Platform Engineer / AWS DevOps · przewodnik po polsku · 17 października 2026

Materiał dla prowadzącego, który nie pracuje na co dzień z DevOps. Wyjaśnia, co Terraform zmienia w AWS, do czego służy state i dlaczego nieudane wdrożenie nie oznacza powrotu do punktu wyjścia. Pytania do kandydata są po angielsku.

Q3 zajmuje 10 minut w 60-minutowej rozmowie. Przeczytaj scenariusze wcześniej; podczas interview zadaj pytanie główne, obowiązkowe dopytanie o częściowy błąd i najwyżej jedno dodatkowe. Przykłady są dydaktyczne, nie opisują potwierdzonej infrastruktury pracodawcy ani osiągnięć kandydata.

Najważniejsze: kandydat powinien umieć przewidzieć skutki zmiany, ograniczyć dostęp do stanu i bezpiecznie wznowić pracę po błędzie.

Spis treści

1. Pytanie główne i jego sens 2
2. Podstawy: kod, AWS i state to trzy różne rzeczy 2
3. Przykład krok po kroku: nowy zbiór orders 3
4. Plan i apply: propozycja oraz wykonanie 3
5. Gdzie trzymać state i co daje blokada 4
6. Scenariusz: dwie osoby wdrażają jednocześnie 4
7. Obowiązkowy scenariusz: bucket powstał, policy nie 5
8. Czy cofnięcie commita cofa infrastrukturę? 5
9. Dev, test i prod: co trzeba rozdzielić 6
10. Drift: ktoś zmienił zasób ręcznie 6
11. Plan chce usunąć zasób z danymi 6
12. Sekrety w state: co znaczy sensitive 7
13. Możliwe odpowiedzi kandydata i właściwe dopytania 7
14. Jak poprowadzić 10 minut i zapisać ocenę 8
15. Ściąga przed zadaniem pytania 9

1. Pytanie główne i jego sens

> “You are extending a data-platform Terraform module with raw and curated buckets, IAM roles, a processing job, and alarms for dev, test, and production. Walk me through the change from pull request to production. Where is the state stored, and how do you prevent two people from making conflicting changes?”

Po polsku: zespół dodaje zasoby platformy danych. Jak kandydat przeprowadzi zmianę przez review, testy i środowiska oraz zapobiegnie kolizjom?

To nie jest pytanie o zapamiętaną składnię. Sprawdzasz trzy umiejętności: powtarzalne wdrażanie, współpracę nad wspólną infrastrukturą oraz rozumienie skutków awarii. W tej roli bucket, rola i zadanie przetwarzające mają wspólnie umożliwić dostarczenie danych, więc samo utworzenie zasobów nie zamyka tematu.

Obowiązkowe dopytanie:

> “An apply creates the bucket but fails while configuring its policy. What would you check before the next run? Does reverting the commit revert the infrastructure?”

Nie pomijaj go na rzecz kilku definicji. Właśnie ten przykład odróżnia znajomość komend od rozumienia rzeczywistych zmian w AWS.

2. Podstawy: kod, AWS i state to trzy różne rzeczy

Terraform jest narzędziem Infrastructure as Code: opisujesz pożądane zasoby w plikach, a narzędzie ustala operacje potrzebne do ich utworzenia lub zmiany. Provider jest integracją, np. z API AWS. Moduł grupuje konfigurację do ponownego użycia, np. bucket, uprawnienia i alarmy dla nowego zbioru danych.

Element	Co zawiera	Przykład
Kod w Git	Pożądany opis infrastruktury	Bucket raw i reguły dostępu
Rzeczywiste AWS	Zasoby istniejące teraz	Utworzony bucket z konkretną nazwą
State	Powiązanie adresów Terraform z zasobami oraz zapis ich atrybutów	Ten element modułu odpowiada temu bucketowi

State nie jest kopią danych z bucketa. Odtworzenie pliku stanu nie przywraca usuniętych plików. Pomaga Terraformowi rozpoznać, czym zarządza; nie zastępuje backupu danych. HashiCorp: cel state.

Przydatne dopytanie:

> “If I delete the state file, what happens to the actual bucket and its data?”

Szukasz odpowiedzi: samo usunięcie stanu nie usuwa bucketa przez API, ale Terraform traci swoje powiązanie. Następny przebieg może proponować nowe zasoby lub zgłosić konflikt. Usunięcie state nie jest metodą naprawy nieudanego wdrożenia.

3. Przykład krok po kroku: nowy zbiór orders

Załóżmy, że istnieje moduł platformy, a kandydat dodaje zbiór orders. Ma powstać miejsce na pliki wejściowe, miejsce na wynik, rola zadania, zadanie przetwarzające i alarm. Nie wymagaj pisania konfiguracji podczas rozmowy.

1. Ustala wymagania: kto zapisuje wejście, kto czyta wynik, jakie dane chronimy i jak sprawdzimy poprawne przetworzenie.
2. Zmienia moduł lub jego parametry w gałęzi Git. Sprawdza też wersję artefaktu zadania i potrzebne uprawnienia.
3. Otwiera pull request. Automatyczne kontrole sprawdzają format, poprawność konfiguracji i uzgodnione reguły projektu. Druga osoba przegląda zmianę.
4. Ogląda plan dla właściwego środowiska: co będzie dodane, zmienione lub usunięte. Sprawdza konto AWS, region i backend, a nie tylko zielony wynik komendy.
5. Wdraża do dev i sprawdza mały plik: wejście, wykonanie, wynik oraz odczyt przez docelową rolę. Potem przechodzi przez test i kontrolowane zatwierdzenie prod.
6. Po wdrożeniu sprawdza alarmy i dane. Zostawia ślad: wersja kodu, wynik planu, wykonane wdrożenie oraz wynik sprawdzenia.

Kolejność szczegółowych bramek może być inna. Oceniaj, czy wdrożona wersja odpowiada przejrzanej zmianie i czy błąd w dev zatrzyma promocję.

> “What would prove the new dataset is usable, rather than just provisioned?”

Dobra odpowiedź mówi o odczycie prawidłowego wyniku przez uprawnionego odbiorcę. Lista istniejących bucketów potwierdza znacznie mniej.

4. Plan i apply: propozycja oraz wykonanie

`terraform plan` pokazuje proponowane operacje. Standardowo odświeża wiedzę o zarządzanych obiektach i porównuje ją z konfiguracją. Nie jest obietnicą, że później wszystko się uda: uprawnienia, limity lub stan środowiska mogą się zmienić. `terraform apply` wykonuje operacje. HashiCorp: plan.

Wypowiedź	Jak ją interpretować
“I check that the plan is green.”	Dopytaj o zawartość planu; sukces komendy to za mało
“I check replacements, deletes, the account and access changes.”	Konkretna kontrola wpływu
“The approved plan is applied, and stale plans are regenerated and reviewed.”	Rozumienie związku review z wykonaniem
“The PR plan always guarantees the production result.”	Nie rozumie ograniczeń prognozy

Zapisany plan można przekazać do apply. Jeśli między review a wdrożeniem zmieniły się warunki, potrzebne może być ponowne planowanie i zatwierdzenie. Nie akceptuj cichego wdrożenia innego zakresu niż ten, który ktoś ocenił.

> “The reviewed plan is two days old and another change has been deployed. What happens before your production apply?”

Oczekuj sprawdzenia aktualności i procedury ponownego review, nie obchodzenia zabezpieczeń po to, by wymusić użycie starego planu.

5. Gdzie trzymać state i co daje blokada

Backend określa miejsce przechowywania stanu. W zespole sensowny jest kontrolowany zdalny backend, np. S3 albo HCP Terraform, zamiast niezależnych kopii na laptopach. Samo słowo „S3” nie dowodzi rozwiązania współbieżności.

W backendzie S3 blokowanie przez `use_lockfile` jest opcjonalne i wymaga konfiguracji oraz odpowiednich uprawnień. Starsze blokowanie oparte na DynamoDB jest oznaczone jako deprecated. Kandydat może znać je z rzeczywistego projektu; poprawny opis takiego środowiska jest wartościowy. HashiCorp: backend S3.

Blokada przypomina zajęcie jednego wspólnego dokumentu do edycji: drugi proces nie powinien równocześnie zmieniać tego samego stanu. Nie rozwiązuje wszystkich konfliktów kodu ani zależności między dwoma osobnymi stanami.

> “What actually takes the lock in your setup, and what happens to the second run?”

Dodatkowo słuchaj o ograniczonym dostępie, szyfrowaniu, historii wersji i sposobie odzyskania state. Backend nie powinien być publicznym plikiem ani zwykłym artefaktem dostępnym wszystkim użytkownikom CI.

6. Scenariusz: dwie osoby wdrażają jednocześnie

Anna i Ben zatwierdzają dwa PR-y. Oba dotyczą produkcji i tego samego stanu. Anna uruchamia apply; chwilę później Ben robi to samo.

Dobra odpowiedź: backend blokuje jednoczesny zapis stanu. Pipeline może dodatkowo kolejkować wdrożenia dla tego środowiska. Po zakończeniu pierwszej zmiany druga jest ponownie oceniana wobec aktualnego stanu. Blokada backendu i kolejka całego workflow pełnią powiązane, lecz różne funkcje.

Jeśli kandydat mówi „wyłączę blokadę”:

> “How would you determine whether the lock belongs to a live deployment?”

`force-unlock` nie jest standardową odpowiedzią na oczekiwanie. Przy osieroconej blokadzie trzeba ustalić właściciela, stan procesu i przyczynę przerwania. Odblokowanie aktywnego wdrożenia może dopuścić kolizję. HashiCorp: locking.

Nie wymagaj od kandydata nazw wszystkich flag. Wymagaj bezpiecznej decyzji, gdy narzędzie mówi, że ktoś już pracuje.

7. Obowiązkowy scenariusz: bucket powstał, policy nie

Apply miał utworzyć bucket i skonfigurować dostęp. Pierwsza operacja się udała, druga kończy się AccessDenied. To **częściowy sukces**, a nie transakcja typu „wszystko albo nic”. Terraform nie cofa automatycznie całego apply. HashiCorp: apply.

Rozsądna sekwencja odpowiedzi:

1. Zachowuję komunikat błędu i ustalam dokładnie nieudaną operację oraz tożsamość.
2. Sprawdzam, co powstało w AWS i co zostało zapisane w state. Nie zakładam, że po błędzie nic nie istnieje ani że cały stan jest nieaktualny.
3. Ustalam przyczynę, np. brak uprawnienia do polityki, jawny deny lub błędną konfigurację.
4. Naprawiam przyczynę w kontrolowany sposób. Oglądam nowy plan dla tego samego konta i środowiska, zwłaszcza operacje usuwania i odtwarzania.
5. Wykonuję dalsze zmiany i sprawdzam dostęp do danych. Jeżeli obiekt istnieje, ale nie ma powiązania w state, osobno wyjaśniam przyczynę i rozważam import.

Import oznacza objęcie istniejącego zasobu zarządzaniem Terraform. Nie jest obowiązkowym krokiem po każdym błędzie: poprawnie zapisany bucket nie potrzebuje ponownego importowania. Nie akceptuj usuwania state „na wszelki wypadek”.

> “What evidence tells you whether the bucket is already managed by Terraform?”

To zmusza do porównania rzeczywistego obiektu z jego adresem w stanie, zamiast zgadywania na podstawie samego błędu pipeline'u.

8. Czy cofnięcie commita cofa infrastrukturę?

Commit jest wersją plików w Git. Jego cofnięcie zmienia opis pożądanego stanu; samo w sobie nie wysyła operacji usuwania lub zmiany do AWS. Potrzebne jest kolejne wykonanie procesu wdrażania, które może uruchomić się automatycznie.

Przykład: nowy kod utworzył bucket. Revert usuwa jego definicję z konfiguracji. Następny plan może proponować usunięcie bucketa. To wymaga oceny danych, zależności i ochrony przed usunięciem. Nie jest bezpiecznym „undo”.

> “What if the new bucket already contains today's source files?”

Dobra odpowiedź: zabezpieczyć dane, ustalić konsumentów i zakres potrzebnej zmiany. Czasami bezpieczniejsza jest mała poprawka do przodu, czasami cofnięcie wersji aplikacji. Decyzja wynika ze skutków, nie z przyzwyczajenia.

Jeśli kandydat mówi „rollback jest automatyczny”, pozwól doprecyzować:

> “Which tool performs that rollback, and what exactly does it restore?”

Może opisywać własny pipeline. Wtedy oceniaj jego mechanizm i ograniczenia, a nie samą nazwę „rollback”. Szczególnie sprawdź, co dzieje się z danymi.

9. Dev, test i prod: co trzeba rozdzielić

Środowiska potrzebują osobnego stanu i kontrolowanej konfiguracji. Wspólny moduł może być używany wielokrotnie, z innymi nazwami, rolami, rozmiarami i kontami. Kopiowanie całej infrastruktury do trzech folderów bez sposobu utrzymania spójności szybko prowadzi do rozjazdów.

Oddzielny state ogranicza zakres jednej operacji, ale nie zastępuje ograniczeń dostępu. Jeśli rola dev może usuwać zasoby produkcyjne, sama nazwa środowiska nie stanowi ochrony. Workspaces rozdzielają stany, lecz nie są automatycznie pełną granicą bezpieczeństwa. HashiCorp: workspaces.

> “What prevents a dev deployment from changing production resources?”

Oczekuj powiązania konta i roli AWS, backendu, parametrów oraz bramek pipeline'u. Osobne konta są dobrym wariantem, ale nie wymagaj go jako jedynej architektury. Oceń rzeczywiste zabezpieczenia opisanego rozwiązania.

Warto zapytać, jak wersjonuje się moduł i providera. Nie musisz sprawdzać konkretnych numerów: ważne, czy aktualizacja zależności jest świadomą zmianą, a nie niespodzianką podczas wdrażania do prod.

10. Drift: ktoś zmienił zasób ręcznie

Drift to rozbieżność między zarządzaną konfiguracją a rzeczywistą infrastrukturą. Przykład: w czasie incydentu ktoś ręcznie zwiększył limit lub zmienił policy.

> “Someone changes a resource in the console. How would you detect drift and decide which configuration to keep?”

Dobra odpowiedź: sprawdzę plan i faktyczną zmianę, ustalę jej właściciela oraz powód. Jeśli zmiana ma zostać, zapiszę ją w kodzie i przejdę odpowiedni review. Jeśli była błędna, odtworzę uzgodnioną konfigurację. Nie zatwierdzam planu w ciemno, bo mógłby usunąć potrzebną poprawkę awaryjną.

Słabsza odpowiedź: „Terraform zawsze ma rację” albo „od tej pory zmieniamy tylko w konsoli”. Obie pomijają decyzję, jaki stan jest pożądany.

Ważna granica: plan dotyczy zasobów i atrybutów objętych zarządzaniem. Nie jest pełnym skanerem wszystkich zmian na całym koncie AWS.

11. Plan chce usunąć zasób z danymi

> “The plan includes deleting a resource that contains data. What would you investigate before proceeding?”

Kandydat powinien zatrzymać wdrożenie i wyjaśnić powód. Może chodzić o celową likwidację, zmianę właściwości wymuszającej odtworzenie albo zmianę adresu zasobu podczas refaktoryzacji modułu.

Przykład: zmieniono nazwę elementu w kodzie, choć fizyczny bucket powinien zostać. W takim przypadku mechanizm `moved` może opisać przeniesienie adresu w stanie. Nie rozwiązuje jednak każdej zmiany wymagającej fizycznej wymiany zasobu. HashiCorp: refaktoryzacja.

Zapytaj też o odbiorców, backup, retencję, odtworzenie i zgodę na utratę zasobu. `prevent_destroy` może blokować pewne operacje, ale nie jest backupem ani uniwersalną ochroną przed każdym sposobem usunięcia danych.

Sygnał dobrego poziomu: kandydat potrafi powiedzieć, kiedy przestaje klikać „approve”, choć pipeline technicznie umożliwia kontynuację.

12. Sekrety w state: co znaczy sensitive

> “Does marking a value sensitive remove it from Terraform state?”

Nie. Oznaczenie ogranicza pokazywanie wartości w zwykłym wyjściu, ale wartość może pozostać w state i zapisanym planie. Dostęp do tych plików trzeba chronić. Mechanizmy ephemeral i write-only mają odrębne zastosowania i ograniczenia; nie wymagaj ich składni na tym poziomie. HashiCorp: dane wrażliwe.

Przykład: hasło nie pojawia się w logu komendy, ale nadal jest dostępne komuś, kto może pobrać niezaszyfrowaną kopię stanu. „Nie widać w konsoli” nie znaczy „nigdzie nie zostało zapisane”.

> “Who can read state and plan artifacts in your project, and how are they protected?”

Szukasz ograniczeń dostępu i rozsądnego obchodzenia się z artefaktami, nie zapewnienia, że dowolne narzędzie do sekretów automatycznie usuwa wszystkie kopie. Nie proś o pokazanie prawdziwego hasła lub pliku stanu z poprzedniego projektu.

13. Możliwe odpowiedzi kandydata i właściwe dopytania

Kandydat mówi	Dopytanie po angielsku	Co rozstrzygasz
“I only extended existing modules.”	“Which resource or permission did you change, and what did the plan show?”	Własny wkład w istniejącym rozwiązaniu
“We used S3 and DynamoDB.”	“How did locking work, and what happened after a failed run?”	Konkretne doświadczenie, nawet ze starszym rozwiązaniem
“We used HCP Terraform.”	“How were workspaces, access and production runs controlled?”	Rozumienie mechanizmu mimo innego backendu
“I applied changes locally.”	“How did the team coordinate state and production access?”	Lokalny CLI może używać dobrego zdalnego backendu
“I never configured the backend.”	“What did you need to know before running a change safely?”	Uczciwa granica odpowiedzialności
“I delete state and try again.”	“How would Terraform recognise resources already created?”	Czy po wyjaśnieniu rozumie skutek

Przykładowa dobra odpowiedź, nie tekst do recytacji:

> “I would review the module change and a plan for the correct environment, deploy to dev, and test the data flow before production approval. Each environment would have controlled remote state with locking. After a failed apply I would inspect the error, actual resources and state, fix the cause, then review a fresh plan. A Git revert changes configuration; it does not automatically undo AWS changes or restore data.”

To wystarcza jako spójny fundament. Większą samodzielność potwierdza prawdziwy przykład błędu, konkretna decyzja i sposób sprawdzenia naprawy.

14. Jak poprowadzić 10 minut i zapisać ocenę

Czas	Zadanie
0-4 min	Pytanie główne: droga zmiany, state i współbieżność
4-7 min	Obowiązkowy częściowy błąd i Git revert
7-9 min	Jedno dopytanie: środowiska, drift, usuwanie lub sekrety
9-10 min	Wyjaśnienie niejasności i zapis dowodu

Proponowana skala pomocnicza, nie oficjalny próg zatrudnienia:

Ocena	Obserwacja
1	Po wyjaśnieniu nadal traktuje apply jak pełne automatyczne undo lub proponuje usuwanie stanu bez analizy
2	Rozumie podstawy, lecz wymaga prowadzenia przy blokadzie, częściowym błędzie lub kontroli wpływu
3	Samodzielnie opisuje typowy bezpieczny proces i sprawdzenie skutków
4	Uzasadnia kompromisy oraz pokazuje konkretny przypadek współbieżności, driftu lub awarii
N/O	Temat nie został dostatecznie sprawdzony

Zapisz: **samodzielna odpowiedź / użyta podpowiedź / dowód z doświadczenia / niewyjaśniona kwestia**. Brak znajomości nowej flagi backendu nie powinien przeważyć nad poprawnym rozumieniem stanu i bezpieczną pracą z produkcją.

15. Ściąga przed zadaniem pytania

Po co pytasz: sprawdzić, czy kandydat bezpiecznie zmienia wspólną infrastrukturę, a nie tylko uruchamia komendy.

Słownik: moduł = zestaw konfiguracji do użycia ponownie; provider = integracja z API; backend = miejsce stanu; state = mapa zarządzanych zasobów; plan = propozycja zmian; apply = wykonanie; lock = blokada równoczesnej zmiany stanu; drift = rozbieżność z rzeczywistością.

Minimum dobrej odpowiedzi:

- Review i plan dla właściwego konta oraz środowiska.
- Zdalny, chroniony state oraz rzeczywista blokada.
- Test danych i dostępu po wdrożeniu.
- Po awarii porównanie błędu, AWS i state; naprawa przyczyny, nowy plan.
- Git revert nie jest automatycznym cofnięciem infrastruktury ani danych.

Jedno najlepsze dopytanie:

> “The bucket already exists after the failed apply. What exactly would you inspect before running Terraform again?”

Sygnały wymagające wyjaśnienia: ślepe force-unlock, kasowanie state, AdminAccess jako pierwsza naprawa, akceptacja usuwania danych bez analizy, traktowanie sensitive jako usunięcia sekretu ze stanu.

Nie wymagaj: całego HCL, projektowania firmowego systemu wdrożeń ani pamięciowej znajomości wszystkich komend. Wymagaj związku między działaniem, jego skutkiem i dowodem poprawności.

Materiały powiązane: plan rozmowy, Q4: CI/CD i OIDC.